

Лабораторна робота №6

Тема: REST API, fetch і swagger.

Мета роботи: Набуття практичних навичок роботи з нативним fetch в Node.js, версіонування, rate limiting та документування API.

1.1. План лабораторної роботи

1. Ознайомитись з короткими теоретичними відомостями.
2. Виконати завдання лабораторної роботи згідно з варіантом.
3. Оформити звіт та надати відповіді на контрольні запитання.

1.2. Теоретичні відомості

REST - архітектурний стиль

REST (Representational State Transfer) - архітектурний стиль побудови розподілених систем на основі HTTP. REST не є протоколом або стандартом це набір обмежень та принципів проектування API:

- Stateless - сервер не зберігає стан клієнта між запитами. Кожен запит містить всю необхідну інформацію для його обробки
- Ресурси - кожна сутність є ресурсом з унікальним URI. Наприклад /items, /items/1
- Уніфікований інтерфейс - HTTP-методи визначають операцію над ресурсом
- Клієнт-сервер - клієнт і сервер незалежні та взаємодіють виключно через API

- Кешованість - відповіді можуть бути кешовані на стороні клієнта

Версіонування API

Версіонування дозволяє вносити зміни в API без порушення роботи існуючих клієнтів. При зміні структури відповіді або поведінки ендпоінту - зміни вносяться в нову версію, стара залишається незмінною. Найпоширеніший підхід - версіонування через URL:

```
/api/v1/items  
/api/v2/items
```

У Fastify версіонування реалізується через параметр `prefix` при реєстрації плагінів. Один набір маршрутів реєструється під різними префіксами без дублювання коду.

Пагінація

Пагінація - механізм розбиття великого набору даних на сторінки. Параметри передаються через `query string`, відповідь містить дані та метадані для навігації:

```
GET /api/v2/items?page=1&limit=10
```

```
{  
  "data": [...],  
  "meta": { "total": 100, "page": 1, "limit": 10, "totalPages": 10 }  
}
```

Rate Limiting

Rate limiting - механізм обмеження кількості запитів від одного клієнта за певний проміжок часу. Захищає сервер від перевантаження та зловмисного використання. При перевищенні ліміту сервер повертає 429 Too Many Requests. Плагін `@fastify/rate-limit` реалізує rate limiting на рівні Fastify з автоматичним додаванням інформаційних заголовків до кожної відповіді.

Заголовки для rate limiting:

Заголовок	Призначення
X-RateLimit-Limit	Максимальна кількість запитів за період
X-RateLimit-Remaining	Кількість запитів що залишилась
Retry-After	Секунди очікування перед повторним запитом

Документація API - Swagger

Swagger (OpenAPI) - стандарт опису REST API у форматі JSON або YAML. У середовищі Node.js він став критично важливим інструментом, оскільки без документації API фактично «не існує» для інших розробників.

Для чого потрібен?

- Він дозволяє розробникам тестувати методи безпосередньо в браузері без використання сторонніх програм типу Postman;
- Документація слугує чітким «контрактом», де фронтенд-розробник заздалегідь знає точну структуру JSON-відповіді, яку надішле сервер;

- На основі OpenAPI-специфікації можна автоматично генерувати клієнтські бібліотеки для різних мов програмування;
- При використанні автоматичної генерації документація завжди відповідає реальному стану коду та поведінці API.

Плагін `@fastify/swagger` генерує OpenAPI-специфікацію автоматично на основі JSON Schema зареєстрованих маршрутів. `@fastify/swagger-ui` відображає інтерактивний інтерфейс за адресою `/docs` що дозволяє тестувати API безпосередньо з браузера. Перевага автоматичної генерації - документація завжди актуальна і відповідає актуальній поведінці API.

Нативний fetch в Node.js

Починаючи з Node.js 18 глобальна функція `fetch` доступна без додаткових залежностей. `fetch` повертає Promise з об'єктом `Response`. На відміну від більшості HTTP-клієнтів, `fetch` не кидає виключення при HTTP помилках (4xx, 5xx) - лише при мережевих помилках. Перевірка `response.ok` є обов'язковою:

```
const response = await fetch('https://api.example.com/data');
if (!response.ok) throw new Error(`HTTP error: ${response.status}`);
const data = await response.json();
```

Retry з exponential backoff

Retry логіка - повторення запиту при тимчасовій недоступності зовнішнього сервісу. Exponential backoff - стратегія збільшення затримки між спробами в геометричній прогресії: 1с, 2с, 4с. Фіксована затримка між

спробами є менш ефективною - при одночасному перевантаженні багатьма клієнтами синхронні `retry` збільшують навантаження на сервіс що відновлюється. `Exponential backoff` розподіляє повторні запити в часі:

```
const fetchWithRetry = async (url, retries = 3) => {
  for (let attempt = 0; attempt < retries; attempt++) {
    try {
      const response = await fetch(url);
      if (!response.ok) throw new Error(`HTTP ${response.status}`);
      return response;
    } catch (error) {
      if (attempt === retries - 1) throw error;
      const delay = 1000 * Math.pow(2, attempt); // 1s, 2s, 4s
      await new Promise(resolve => setTimeout(resolve, delay));
    }
  }
};
```

Timeout через AbortController

`AbortController` - вбудований API для скасування асинхронних операцій. Використовується для встановлення `timeout` на `fetch`-запити - якщо зовнішній сервер не відповів за визначений час, запит скасовується і звільняються ресурси:

```
const fetchWithTimeout = async (url, timeout = 5000) => {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), timeout);
  try {
    return await fetch(url, { signal: controller.signal });
  } finally {
    clearTimeout(timer);
  }
};
```

Кешування відповідей зовнішнього сервісу

Кешування зменшує кількість запитів до зовнішнього сервісу та прискорює відповідь клієнту. Відповідь зберігається разом з часом запису. При наступному запиті перевіряється актуальність кешу через TTL - якщо час з моменту останнього запиту менший за TTL, дані повертаються з кешу без нового запиту до зовнішнього сервісу.

Graceful Degradation

Graceful degradation - підхід до проектування систем при якому відмова залежного сервісу не призводить до повної відмови основного. При недоступності зовнішнього сервісу система повертає частково заповнену відповідь замість повідомлення про помилку. Це забезпечує безперервність роботи основного функціоналу незалежно від стану зовнішніх залежностей.

1.3. Завдання до лабораторної роботи

Ознайомтесь перед виконанням з "Теоретичними відомостями". Завдання є продовженням попередніх лабораторних робіт. Рекомендується ознайомитись з усіма пунктами перед початком виконання.

1. Створіть гілку Lab_6 з Lab_5. Код і індивідуальний варіант з попередньої роботи.

2. Версіонування API.

Перенесіть всі існуючі маршрути під префікс /api/v1 використовуючи механізм prefix при реєстрації плагінів у Fastify.

3. Коректні HTTP статус-коди.

Впевніться що всі ендпоінти повертають семантично правильні коди відповіді.

4. Пагінація для /api/v2/items.

Реалізуйте нову версію GET /api/v2/items з підтримкою пагінації. /api/v1/items залишається без змін:

- Параметри запиту: ?page=1&limit=10. При відсутності параметрів застосовувати значення за замовчуванням.
- Відповідь містить масив даних та метадані: total, page, limit, totalPages

5. Rate Limiting.

Підключіть @fastify/rate-limit для обмеження кількості запитів:

- Глобальне обмеження, не більше 100 запитів на хвилину з однієї IP-адреси;
- При перевищенні ліміту повертати 429 Too Many Requests;

6. Документація API.

Підключіть @fastify/swagger та @fastify/swagger-ui. Документація доступна за адресою GET /docs. Переконайтесь що всі маршрути які ви реалізовували в попередніх роботах присутні в документації.

7. Інтеграція із зовнішнім сервісом через fetch.

Встановіть json-server як “dev” залежність. Створіть файл db.json з даними згідно з варіантом:

- Variant 1 Inventory - категорії товарів: [{ "id": 1, "name": "Electronics", "tax": 0.2 }]
- Variant 2 Todo - пріоритети: [{ "id": 1, "name": "high", "maxDays": 1 }]

- Variant 3 Books - жанри: [{ "id": 1, "name": "Fiction", "ageRating": 12 }]
- Variant 4 Students - курси: [{ "id": 1, "name": "Computer Science", "credits": 240 }]
- Variant 5 Smart Home - типи пристроїв: [{ "id": 1, "type": "lamp", "powerWatt": 10 }]

Додати скрипт в package.json - "external": "json-server db.json --port 3001". Запускати в окремому терміналі командою npm run external паралельно з основним сервером.

Реалізуйте ендпоінт GET /api/v1/items/:id/details:

- Отримати запис з файлового сховища (репозиторій метод)
- Зробити fetch до JSON Server, об'єднати дані та повернути.

Вимоги до реалізації fetch:

- Retry логіка з exponential backoff - до 3 спроб із затримкою 1с, 2с, 4с
- Timeout через AbortController - 5 секунд
- Кешування відповіді у файл data/cache/reference.json з TTL 120 секунд.
- Отримувати дані з “кешу” якщо TTL активний.
- При недоступності зовнішнього сервісу після використання всіх спроб - повернути запис з основними даними, поля що мали бути зовнішніми даними встановити як null.

8. Інтеграція з публічним GitHub API

Реалізуйте аналітичний сервіс, окремий ендпоінт який приймає шлях до репозиторію ?repo=fastify/fastify і повертає **5 інших репозиторіїв, які мають найбільшу кількість спільних “contributors”!!!**. У вас повинні

бути дві версії де функціонал ідентичний, реалізація різна. Опишіть вашу реалізацію для цього завдання в звіті.

Вимоги:

- 1) Реалізація v1. Використовувати лише REST API github.
/api/v1/github/shared-repos
- 2) Реалізація v2. Можна використовувати як REST API github так і GraphQL API. */api/v2/github/shared-repos*
- 3) Продемонструвати роботу з репозиторіями де є >20, >100, >300 “contributors”.

9. Перевірте роботу пагінації, rate limiting, документацію в /docs та поведінку */api/v1/items/:id/details* при зупиненому JSON Server.

Контрольні запитання

1. Що таке REST і які його основні принципи?
2. Навіщо потрібне версіонування API?
3. Що таке пагінація і які параметри вона використовує?
4. Навіщо потрібен rate limiting?
5. Що таке exponential backoff і чим він кращий за фіксовану затримку між retry?
6. Навіщо потрібен AbortController при роботі з fetch?
7. Чому fetch не кидає виключення при статусах 4xx та 5xx?
8. Що таке TTL при кешуванні і навіщо він потрібен?
9. Що таке graceful degradation і коли його застосовують?

Вимоги до оформлення звіту

1. Титульна сторінка
2. Мета лабораторної роботи
3. Умова завдання (із зазначенням варіанту)
4. Фрагменти програмного коду
5. Результати виконання
6. Посилання на гілку Lab_6
7. Відповіді на контрольні запитання
8. Висновки